

Problem Set 5

E. Minimum spanning tree for each edge

TIANZHEYU SETTINGS LISTS BLOG TEAMS SUBMISSIONS CONTESTS

☒ show unofficial

tianzheyu submissions

tianzheyu submissions							
#	When	Who	Problem	Lang	Verdict	Time	Memory
381679871	Jul/08/2026 16:10 UTC+10	tianzheyu	E - Minimum spanning tree for each edge	C++23 (GCC 14-64, msys2)	Accepted	1625 ms	161100 KB

<https://codeforces.com/submissions/tianzheyu#>

Process

The goal is to find the weight of the Minimum Spanning Tree (MST) that forces the inclusion of a specific edge, repeated for every edge in the graph.

The optimal approach is to first construct a global MST using Kruskal's algorithm with DSU. And for each edge, if that edge is already in MST the result will just be the total weight of that MST. However, if it is not, then we have to add that edge (u v) into the MST and will have a cycle as a result. In order to change that graph to a tree, we need to remove an edge in the path from u to v in the MST. And since we need minimum total weight, we can only remove the edge with largest weight. Now the question turns into finding the maximum weighted edge in the path from u to v in a tree.

To answer this efficiently across potentially $2 * 10^5$ queries, we can utilize the Binary Lifting. By running a single DFS on the constructed MST, we can precompute a table that stores the 2^i ancestor for each node, alongside the maximum edge weight on the path to that ancestor. This allows us to find the LCA and extract the maximum path weight simultaneously in $O(\log N)$ time per query, bringing the total time complexity to a highly optimal $O(E \log E + E \log V)$.

Challenges and Overcoming

Initially, I attempted to use a global DFS to check for cycles before adding a new edge during Kruskal's algorithm. This approach yielded an $O(E * V)$ time complexity, which is fundamentally unscalable for the given constraints and caused severe Time Limit Exceeded errors. I refactored the cycle detection to use a standard DSU with path compression, bringing the operation down to near $O(1)$ and perfectly resolving the bottleneck.

I originally used a `set<Edge>` to track which edges were successfully included in the MST. However, my overloaded `<` operator only compared the `weight` property. Because `std::set` relies exclusively on this operator to determine element uniqueness (`!(a < b) && !(b < a)`), it silently discarded entirely different edges that simply happened to share the same weight. I completely bypassed this trap by assigning a unique `id` to each edge upon input and managing MST inclusion via a fast `vector<bool> in_mst` array.